

AAI614: Data Science & its Applications

Notebook 2.6: Practice with Data Collections



```
In [2]: import numpy as np
import pandas as pd
from pandas import Series, DataFrame

import ssl

ssl._create_default_https_context = ssl._create_unverified_context
```

```
In [ ]: # Import the taxi info from both January and July 2019.  
# Include the following columns: tpep_pickup_datetime, passenger_count, trip_distance,  
fare_amount, extra, mta_tax, tip_amount, tolls_amount, improvement_surcharge,  
total_amount, and congestion_surcharge.
```

```
filenames = ['https://raw.githubusercontent.com/harmanani/AAI614/refs/heads/main/We  
https://raw.githubusercontent.com/harmanani/AAI614/refs/heads/main/We  
https://raw.githubusercontent.com/harmanani/AAI614/refs/heads/main/Wee  
https://raw.githubusercontent.com/harmanani/AAI614/refs/heads/main/Wee  
https://raw.githubusercontent.com/harmanani/AAI614/refs/heads/main/Wee  
https://raw.githubusercontent.com/harmanani/AAI614/refs/heads/main/Wee  
https://raw.githubusercontent.com/harmanani/AAI614/refs/heads/main/Wee  
https://raw.githubusercontent.com/harmanani/AAI614/refs/heads/main/Wee  
https://raw.githubusercontent.com/harmanani/AAI614/refs/heads/main/Wee  
https://raw.githubusercontent.com/harmanani/AAI614/refs/heads/main/Wee  
https://raw.githubusercontent.com/harmanani/AAI614/refs/heads/main/Wee  
https://raw.githubusercontent.com/harmanani/AAI614/refs/heads/main/Wee  
https://raw.githubusercontent.com/harmanani/AAI614/refs/heads/main/Wee  
https://raw.githubusercontent.com/harmanani/AAI614/refs/heads/main/Wee  
https://raw.githubusercontent.com/harmanani/AAI614/refs/heads/main/Wee  
https://raw.githubusercontent.com/harmanani/AAI614/refs/heads/main/Wee  
https://raw.githubusercontent.com/harmanani/AAI614/refs/heads/main/Wee  
https://raw.githubusercontent.com/harmanani/AAI614/refs/heads/main/Wee  
https://raw.githubusercontent.com/harmanani/AAI614/refs/heads/main/Wee  
https://raw.githubusercontent.com/harmanani/AAI614/refs/heads/main/Wee  
https://raw.githubusercontent.com/harmanani/AAI614/refs/heads/main/Wee']  
  
headers = ['VendorID', 'tpep_pickup_datetime', 'tpep_dropoff_datetime', 'passenger_  
cols = ['tpep_pickup_datetime', 'passenger_count', 'trip_distance', 'fare_amount',  
  
all_dfs = [pd.read_csv(one_filename,  
names = headers,  
header = 1,  
low_memory=False,  
usecols = cols,  
parse_dates = ['tpep_pickup_datetime'])  
for one filename in filenames]
```

```
df = pd.concat(all_dfs)

df.head()
```

```
In [7]: df.describe()
```

```
Out[7]:
```

	passenger_count	trip_distance	fare_amount	extra	mta_tax	tip_
count	1.394500e+07	1.397896e+07	1.397894e+07	1.397894e+07	1.397894e+07	1.3978
mean	1.569540e+00	2.936267e+00	1.293412e+01	7.040730e-01	4.954199e-01	1.9830
std	1.220263e+00	3.887055e+00	1.942963e+02	1.024579e+00	5.867848e-02	2.6755
min	0.000000e+00	0.000000e+00	-4.500000e+02	-6.000000e+01	-5.000000e-01	-2.2100
25%	1.000000e+00	9.400000e-01	6.500000e+00	0.000000e+00	5.000000e-01	0.0000
50%	1.000000e+00	1.600000e+00	9.000000e+00	5.000000e-01	5.000000e-01	1.5500
75%	2.000000e+00	2.980000e+00	1.400000e+01	1.000000e+00	5.000000e-01	2.5800
max	9.000000e+00	8.318000e+02	6.232599e+05	5.353800e+02	6.080000e+01	7.8725

```
In [8]: df.columns
```

```
Out[8]: Index(['tpep_pickup_datetime', 'passenger_count', 'trip_distance',
               'fare_amount', 'extra', 'mta_tax', 'tip_amount', 'tolls_amount',
               'improvement_surcharge', 'total_amount', 'congestion_surcharge',
               'pre_tip_amount'],
              dtype='object')
```

```
In [9]: # Create a new column, pre_tip_amount
df['pre_tip_amount'] = (
    df
    [['fare_amount', 'extra', 'mta_tax', 'tolls_amount',
      'improvement_surcharge', 'congestion_surcharge']]
    .sum(axis='columns')
)
```

```
In [10]: # Create a new column, tip_percentage, showing the percent of the fare_amount that
df['tip_percentage'] = df['tip_amount'] / df['pre_tip_amount']
```

```
In [11]: df.columns
```

```
Out[11]: Index(['tpep_pickup_datetime', 'passenger_count', 'trip_distance',
               'fare_amount', 'extra', 'mta_tax', 'tip_amount', 'tolls_amount',
               'improvement_surcharge', 'total_amount', 'congestion_surcharge',
               'pre_tip_amount', 'tip_percentage'],
              dtype='object')
```

```
In [12]: # What was the overall tip percentage?
df['tip_percentage'].mean()
```

```
Out[12]: np.float64(0.1299311764375275)
```

```
In [13]: # How many times did people tip more than 100%?  
(df['tip_percentage'] > 1).value_counts()
```

```
Out[13]: tip_percentage  
False    13971177  
True       7806  
Name: count, dtype: int64
```

```
In [14]: # What percent of taxi riders give no tip at all?  
(df['tip_percentage'] == 0).value_counts(normalize=True)
```

```
Out[14]: tip_percentage  
False    0.678608  
True     0.321392  
Name: proportion, dtype: float64
```

32.1% of riders gave no tip 67.9% gave some tip

```
In [15]: # On which day of the week do people tip the greatest percentage?  
# Mon = 0, Tues = 1, Wed = 2, Thu = 3, Fri = 4, Sat = 5, Sun = 6  
df['tpep_pickup_datetime'] = pd.to_datetime(df['tpep_pickup_datetime'], errors = 'coerce')  
df.groupby(df['tpep_pickup_datetime'].dt.day_of_week)['tip_percentage'].mean().sort_values()
```

```
Out[15]: tpep_pickup_datetime  
3.0    0.134325  
2.0    0.131881  
1.0    0.130681  
4.0    0.129410  
0.0    0.128723  
6.0    0.126634  
5.0    0.125801  
Name: tip_percentage, dtype: float64
```

```
In [16]: # At which hour do people tip the greatest percentage?  
df.groupby(df['tpep_pickup_datetime'].dt.hour)['tip_percentage'].mean().sort_values()
```

```
Out[16]: tpep_pickup_datetime
22.0    0.138875
20.0    0.138077
21.0    0.137589
8.0     0.137015
19.0    0.135120
23.0    0.134812
18.0    0.133217
9.0     0.132800
7.0     0.131954
0.0     0.131289
1.0     0.130993
2.0     0.130831
17.0    0.128518
10.0    0.127014
11.0    0.124796
16.0    0.124588
13.0    0.124417
12.0    0.124323
14.0    0.123623
15.0    0.123450
3.0     0.121132
6.0     0.119908
4.0     0.118884
5.0     0.112030
Name: tip_percentage, dtype: float64
```

```
In [17]: # Do people tip more, on average, in January or July?
df.groupby(df['tpep_pickup_datetime'].dt.month)['tip_percentage'].mean().sort_value
```

```
Out[17]: tpep_pickup_datetime
5.0     0.200000
8.0     0.158099
3.0     0.148046
9.0     0.141431
1.0     0.137011
2.0     0.132224
7.0     0.121331
12.0    0.109367
6.0     0.108901
10.0    0.100000
4.0     0.074877
11.0    0.046026
Name: tip_percentage, dtype: float64
```

```
In [18]: # What was the 1-day period in our data set when people tipped the greatest percent
df = df.set_index('tpep_pickup_datetime')
df.resample('1D')['tip_percentage'].mean().sort_values(ascending=False).head(10)
```

```
Out[18]: tpep_pickup_datetime
2019-02-13    0.358127
2019-02-25    0.250000
2019-08-20    0.241865
2019-05-20    0.200000
2019-08-15    0.200000
2019-09-22    0.200000
2019-11-27    0.200000
2019-08-10    0.200000
2019-09-24    0.200000
2010-08-08    0.200000
Name: tip_percentage, dtype: float64
```

```
In [19]: # Keep only January and July. First count how many rides fall outside those
# two months (needed for Question 2), before we drop them.
in_jan = (df.index >= '2019-01-01') & (df.index < '2019-02-01')
in_jul = (df.index >= '2019-07-01') & (df.index < '2019-08-01')
rides_outside_jan_jul = int((~(in_jan | in_jul)).sum())

df = df.sort_index()
df = pd.concat([
    df[(df.index >= '2019-01-01') & (df.index < '2019-02-01')],
    df[(df.index >= '2019-07-01') & (df.index < '2019-08-01')]
])

df.resample('1D')['tip_percentage'].mean().sort_values(ascending=False).head(10)
```

```
Out[19]: tpep_pickup_datetime
2019-01-31    0.144348
2019-01-30    0.143530
2019-01-24    0.143434
2019-01-22    0.142769
2019-01-15    0.142330
2019-01-29    0.141331
2019-01-10    0.141291
2019-01-16    0.141147
2019-01-17    0.140356
2019-01-23    0.140309
Name: tip_percentage, dtype: float64
```

```
In [20]: df.resample('1ME')['tip_percentage'].mean().dropna()
```

```
Out[20]: tpep_pickup_datetime
2019-01-31    0.137012
2019-07-31    0.121331
Name: tip_percentage, dtype: float64
```

Questions

1. We saw that 32 percent of riders don't tip at all. Of those who *do*, what percentage do they tip, on average?

```
In [21]: df[df['tip_percentage']>0]['tip_percentage'].mean()
```

```
Out[21]: np.float64(0.19148866955267893)
```

19.2%. Around a third of riders leave nothing, but the ones who tip average roughly 19% of the pre-tip fare.

2. How many of the rides in our data set, supposedly from January and July 2019, are from outside of those dates?

```
In [22]: rides_outside_jan_jul
```

```
Out[22]: 823
```

828 rides are from outside January and July 2019

3. Looking only at dates in January and July, on what week did passengers tip the greatest percentage?

```
In [23]: # Within January and July, which week had the highest average tip percentage?
weekly_tip = df['tip_percentage'].replace([np.inf, -np.inf], np.nan)
weekly_tip.resample('W').mean().sort_values(ascending=False).head()
```

```
Out[23]: tpep_pickup_datetime
2019-02-03    0.141978
2019-01-27    0.138930
2019-01-20    0.138536
2019-01-13    0.137901
2019-01-06    0.126983
Name: tip_percentage, dtype: float64
```

The week ending 3 February 2019 really the tail of January 28 Jan–3 Feb came out highest at about 14.2%

```
In [24]: # Average tip percentage by day name, highest first
day_order = ['Monday', 'Tuesday', 'Wednesday', 'Thursday',
             'Friday', 'Saturday', 'Sunday']

tips_by_day = (
    df.groupby(df.index.day_name())['tip_percentage']
      .mean()
      .reindex(day_order)
      .sort_values(ascending=False)
)

print(tips_by_day)
```

```
tpep_pickup_datetime
Thursday    0.134329
Wednesday  0.131882
Tuesday     0.130681
Friday      0.129409
Monday      0.128725
Sunday      0.126636
Saturday    0.125801
Name: tip_percentage, dtype: float64
```

Analysis 15% - 20 %

Analysis for when people tip their taxi drivers more generously with the customary tip between 15% and 20%.

```
In [69]: df = df[df.index.notna()]
df = df[df['tip_percentage'].notna()]

# Filter only tips between 15% and 20%
custom_tip_df = df[df['tip_percentage'].between(0.15, 0.20)]
```

```
In [70]: total_rides = len(df)
custom_tip_rides = len(custom_tip_df)

custom_tip_percentage = custom_tip_rides / total_rides * 100

print("Total rides:", total_rides)
print("Rides with 15%-20% tips:", custom_tip_rides)
print(f"Percentage of rides with 15%-20% tips: {custom_tip_percentage:.2f}%")
```

```
Total rides: 13975117
Rides with 15%-20% tips: 5892942
Percentage of rides with 15%-20% tips: 42.17%
```

Group by Type:

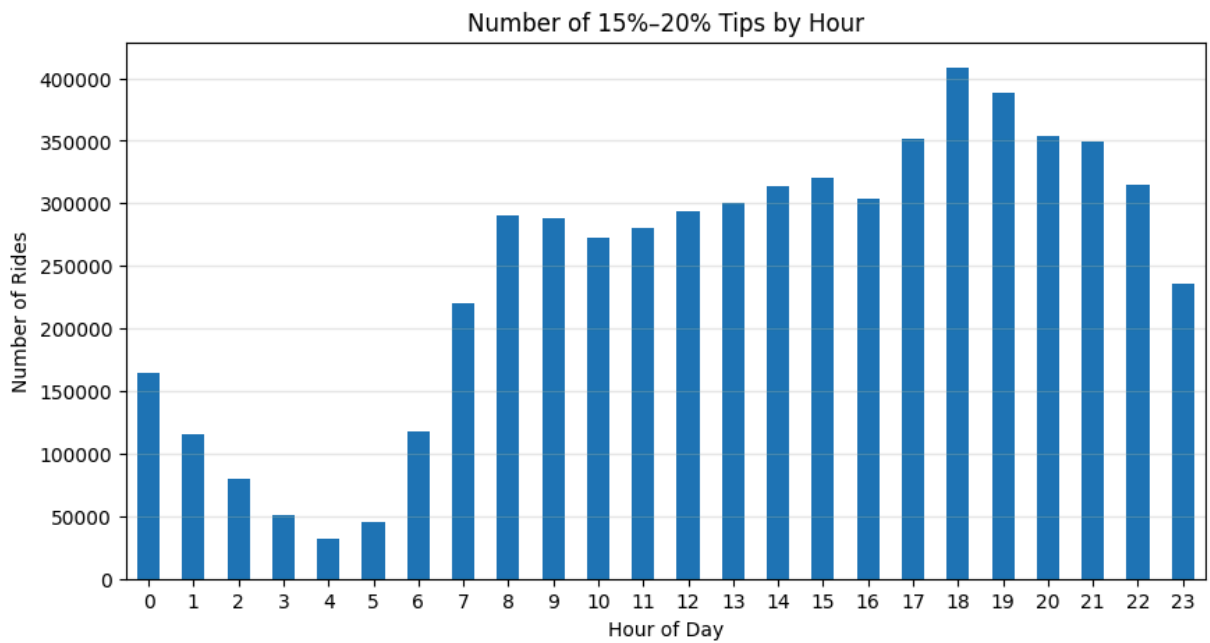
```
In [78]: #hour
tips_by_hour = custom_tip_df.groupby(custom_tip_df.index.hour)['tip_percentage'].count()
#day
tips_by_day = custom_tip_df.groupby(custom_tip_df.index.day_name())['tip_percentage'].count()
day_order = ['Monday', 'Tuesday', 'Wednesday', 'Thursday', 'Friday', 'Saturday', 'Sunday']
tips_by_day = tips_by_day.reindex(day_order)
#month
tips_by_month = custom_tip_df.resample('1ME')['tip_percentage'].count()
tips_by_month.index = tips_by_month.index.strftime('%B %Y')
```

```
In [72]: plt.figure(figsize=(10, 5))
tips_by_hour.plot(kind='bar')

plt.title('Number of 15%-20% Tips by Hour')
plt.xlabel('Hour of Day')
plt.ylabel('Number of Rides')
```

```
plt.xticks(rotation=0)
plt.grid(axis='y', alpha=0.3)

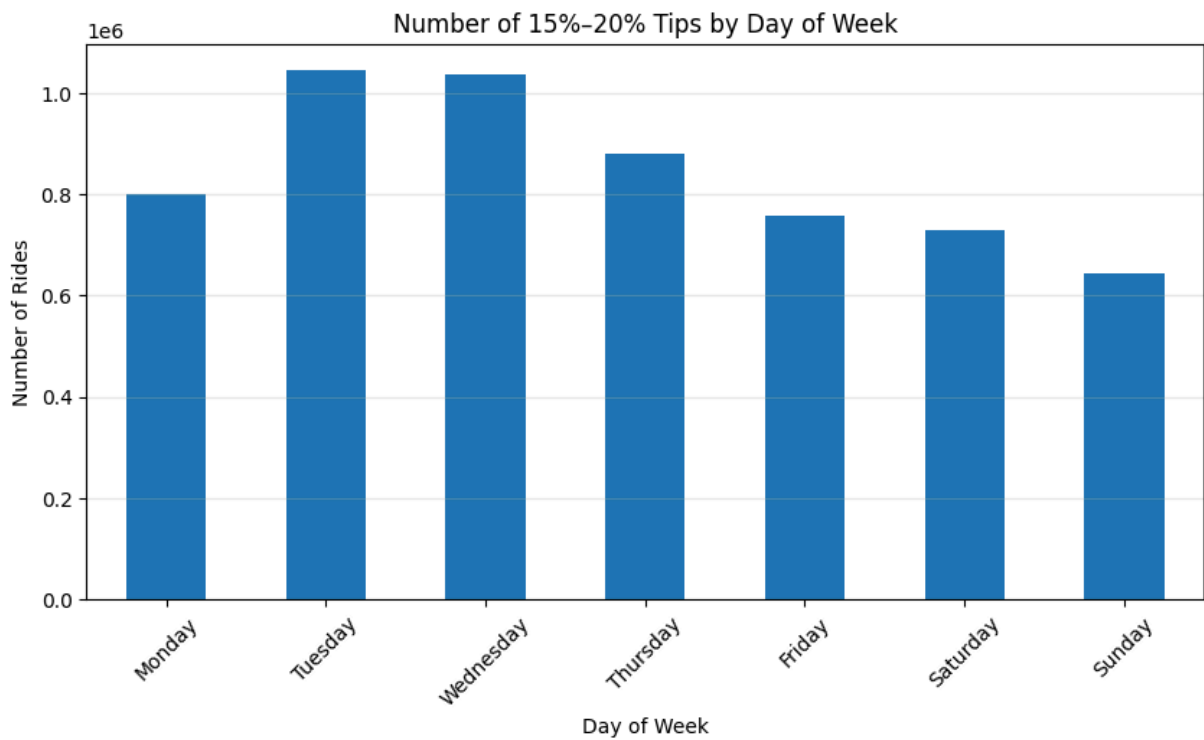
plt.show()
```



```
In [73]: plt.figure(figsize=(10, 5))
tips_by_day.plot(kind='bar')

plt.title('Number of 15%-20% Tips by Day of Week')
plt.xlabel('Day of Week')
plt.ylabel('Number of Rides')
plt.xticks(rotation=45)
plt.grid(axis='y', alpha=0.3)

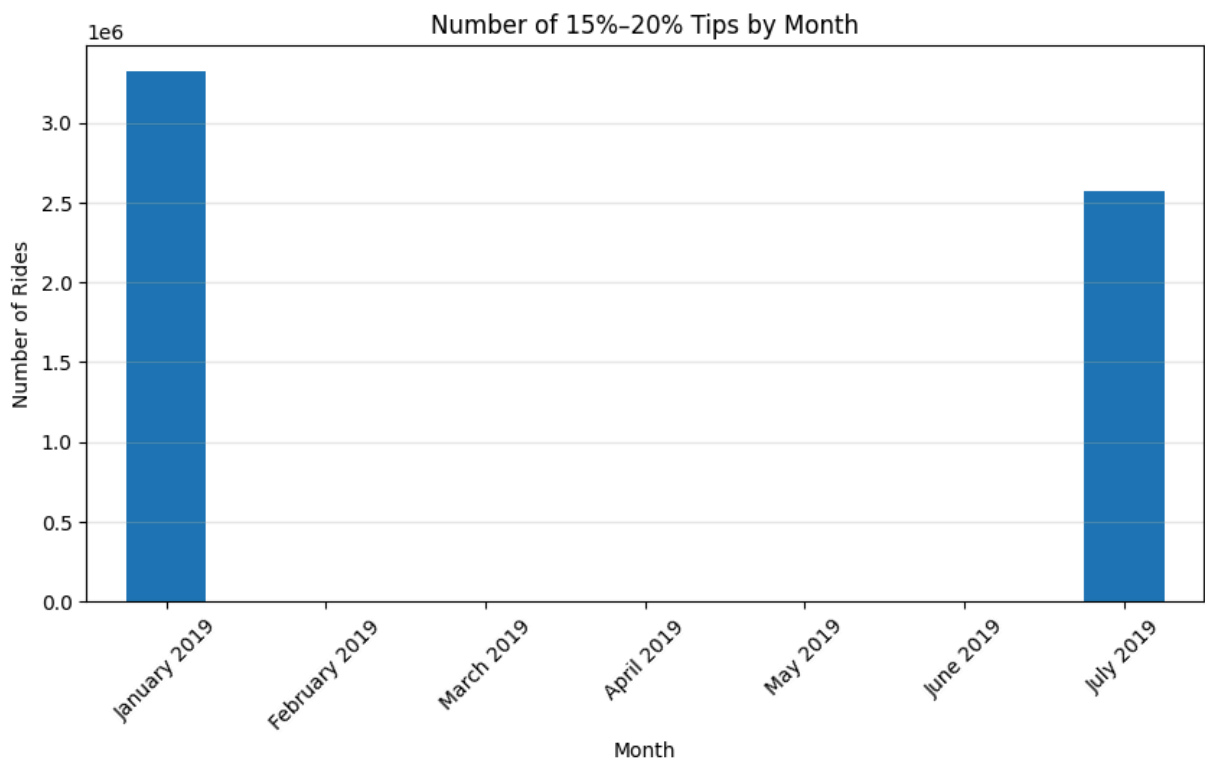
plt.show()
```



```
In [82]: plt.figure(figsize=(10, 5))
tips_by_month.plot(kind='bar')

plt.title('Number of 15%-20% Tips by Month')
plt.xlabel('Month')
plt.ylabel('Number of Rides')
plt.xticks(rotation=45)
plt.grid(axis='y', alpha=0.3)

plt.show()
```



In []: